

Model-Based Deep Learning Project

Recording Preparation Summary

Hebrew + English summary of the project status so far

Prepared after completing the classical baselines and the main synthetic deep-unrolling comparison

חלק א' - סיכום בעברית

1. תמונת מצב קצרה

הפרויקט עוסק ב-Deep Unfolding עבור בעיית sparse recovery. נקודת המוצא היא אלגוריתם ISTA לבעיית LASSO, והמטרה היא לבדוק כיצד ניתן להפוך איטרציות של אופטימיזר קלאסי לרשת נלמדת, תוך שמירה על מבנה מודלי של הבעיה.

עד עכשיו בנינו תשתית ניסויית מסודרת עבור sparse recovery סינתטי, תיקנו את אתחול LISTA כך שיתחיל ממש כמו ISTA, ניקינו את Notebook 02 כך שיהיה ניסוי מרכזי נקי, ומיזגנו את השינויים ל-main. בשלב הבא נרצה לעבור לניסוי על מרחב התכנון של ISTA-unfolding: שיטות אימון שונות ומודלים חדשים שנלמד בהם רק תתי-קבוצות של פרמטרי ISTA.

2. בעיית הבסיס

אנחנו עובדים עם מודל מדידה לינארי:

$$b = A x + \varepsilon$$

כאשר x הוא אות דליל, A היא מטריצת המדידה, ו- ε הוא רעש במדידות. במקרה ללא רעש, $\varepsilon=0$. המטרה היא לשחזר את x מתוך המדידות b .

הבעיה הקלאסית מנוסחת דרך LASSO:

$$\min_x 0.5 \|b - A x\|^2 + \lambda \|x\|_1$$

ISTA פותר את הבעיה הזו באיטרציות של צעד גרדיאנט ולאחריו soft-thresholding:

$$x^{k+1} = S_{\{\lambda/L\}}(x^k + (1/L) A^T(b - A x^k))$$

3. איך זה מתחבר לקורס

הקורס מדבר על הספקטרום בין שיטות model-based לבין deep learning טהור. בפרויקט שלנו:

- **ISTA/FISTA** הם הקצה המודל-בייסדי: אין למידה, רק אלגוריתם קלאסי.
- **LISTA** לוקח את ISTA, פורס אותו למספר שכבות, ולומד מטריצות וספים.
- **ALISTA** משאיר יותר מבנה מודלי: מטריצת תיקון מחושבת אנליטית, ורק מעט סקלרים נלמדים.
- **HyperLISTA** מייצג גרסה עוד יותר קומפקטית, שבה מעט מאוד פרמטרים שולטים בהתנהגות אדפטיבית.

לכן הפרויקט לא רק משווה רשתות, אלא בודק את השאלה המרכזית ב-MBDL: כמה מבנה מודלי כדאי לשמר, וכמה גמישות נלמדת כדאי להוסיף.

4. מה עשינו עד עכשיו

Notebook 01 - Classical Sparse Recovery Baselines

המטרה של Notebook 01 היא לבנות baseline נקי. הרצנו ISTA ו-FISTA על בעיית sparse recovery סינתטית. בנוסף, בדקנו רגישות ל- λ כדי לוודא שהפרמטרים של ISTA/FISTA סבירים. הנוטבוק הזה חשוב כי הוא מגדיר את נקודת ההשוואה: מה ניתן להשיג בלי למידה בכלל.

Notebook 02 - Main Synthetic Deep-Unrolling Comparison

Notebook 02 הוא הניסוי המרכזי הראשון של MBDL. הוא משווה בין ISTA/FISTA לבין LISTA, ALISTA ו-HyperLISTA על אותה בעיית sparse recovery סינתטית. ניקינו אותו כך ש:

- כל המודלים משתמשים באותו $LAM = 0.1$.
- LISTA מאותחל לפי ISTA.
- האימון המרכזי של LISTA משתמש ב-final loss בלבד, כלומר $intermediate_weight = 0.0$.
- התוצאות וה-checkpoints נשמרים בתיקייה ייעודית של Notebook 02.

תיקון LISTA

עדכנו את LISTA כך שגם threshold ההתחלתי שלו יהיה זהה ל-ISTA:

$$\theta_0 = \lambda / L$$

מכיוון שבקוד $\theta = \text{softplus}(\log_theta)$, השתמשנו ב-inverse softplus כדי להגדיר:

$$\log_theta = \text{softplus}^{-1}(\lambda/L)$$

כך מתקבל בפועל $\theta = \lambda/L$, ולכן LISTA מתחיל מאותה נקודת מוצא אלגוריתמית כמו ISTA.

5. תוצאות Notebook 02

מודל	NMSE dB	זמן ריצה משוער	מספר פרמטרים נלמדים	פירוש
ISTA	-5.306	ms 8.132	0	Baseline קלאסי
FISTA	-10.959	ms 7.895	0	Baseline קלאסי מואץ
LISTA	-20.866	ms 8.576	6,000,016	הרבה גמישות נלמדת
ALISTA	-29.763	ms 18.633	32	מעט פרמטרים + מבנה מודלי חזק
HyperLISTA	-61.386	ms 47.034	3	מעט מאוד פרמטרים, ביצועים מצוינים בסינתטי

המסר המרכזי מהטבלה

המעבר מ-ISTA/FISTA למודלים unfolded משפר משמעותית את איכות השחזור. בנוסף, ALISTA ו-HyperLISTA מראים שלא חייבים מיליוני פרמטרים כדי לשפר ביצועים; לפעמים מבנה מודלי נכון מאפשר ללמוד מעט מאוד פרמטרים ועדיין לקבל תוצאה חזקה.

למה LISTA חלש בשכבות האמצע ומשתפר בסוף?

ב-02 Notebook אימנו את LISTA עם final loss בלבד:

$$\text{Loss} = ||x - \hat{x}^{\{16\}}||^2$$

לכן אין דרישה מפורשת שכל שכבת ביניים תיתן שחזור טוב. השכבות הפנימיות יכולות ללמוד ייצוגים שמועילים בעיקר לפלט הסופי. לעומת זאת ALISTA ו-HyperLISTA יותר constrained ומבוססי-אלגוריתם, ולכן כל שכבה שלהם נראית יותר כמו צעד refinement אמיתי.

למה ALISTA/HyperLISTA איטיים יותר למרות שיש להם פחות פרמטרים?

מספר פרמטרים אינו זהה לזמן ריצה. LISTA משתמש בעיקר בכפלי מטריצות צפופים, שהם מאוד אופטימליים ב-PyTorch/GPU ו-HyperLISTA כוללים פעולות כגון residual updates, thresholding מותאם, support selection, וחישובים אדפטיביים. לכן הם parameter-efficient, אך לא בהכרח runtime-efficient.

6. המדדים המרכזיים

מדד	משמעות	למה הוא חשוב
NMSE	שגיאת שחזור מנורמלת: $ x - \hat{x} ^2 / x ^2$	מדד איכות השחזור המרכזי
NMSE dB	10 log10(NMSE)	ערך שלילי יותר הוא טוב יותר
NMSE per layer	NMSE אחרי כל שכבה/איטרציה	מראה קצב התכנסות והתנהגות פנימית
Runtime	זמן inference	בודק יעילות חישובית בפועל
Number of parameters	כמה פרמטרים נלמדים	בודק את רמת ה-data-driven מול model-based

7. מה נאמר למרצה ומה הוא החזיר

עדכנו את המרצה שהפרויקט עוסק ב-Deep Unrolling עבור sparse recovery, ובהמשך תמונות. הצגנו תוצאות ראשוניות עבור ISTA, FISTA, LISTA, ALISTA ו-HyperLISTA. המרצה אישר שימוש ב-MNIST/FashionMNIST וציין שאין צורך ב-DCT/Wavelets כי הדאטה sparse מאוד ב-pixel domain. בנוסף, הוא המליץ להשוות שיטות אימון שונות עבור unfolded optimizers.

לכן ההמשך שלנו צריך להתמקד בשני כיוונים:

1. השוואת שיטות אימון ל-unfolded optimizers.

2. מעבר ל-FashionMNIST/MNIST ב-pixel domain עם חלוקה מתודולוגית נכונה ל-/train/validation .test

8. התכנון הבא

Notebook 03 - ISTA Unfolding Design Space

התכנון הוא להפוך את Notebook 05 הנוכחי לניסוי השלישי המרכזי, עם שם בסגנון:

03_ista_unfolding_design_space.ipynb

מטרתו תהיה להראות שאנחנו לא רק מריצים מודלים קיימים, אלא חוקרים אילו רכיבים מתוך ISTA כדאי ללמוד.

שלושה מודלים חדשים שלנו

מודל	מה לומדים	נוסחה רעיונית	פרמטרים עבור $K=16$
ThresholdISTA	רק threshold לכל שכבה	$x^{k+1} = S_{\{\theta_k\}}(x^k + (1/L)A^T(b - Ax^k))$	16
StepISTA	רק step size לכל שכבה	$x^{k+1} = S_{\{\lambda\gamma_k\}}(x^k + \gamma_k A^T(b - Ax^k))$	16
StepThresholdISTA	+ step size threshold	$x^{k+1} = S_{\{\theta_k\}}(x^k + \gamma_k A^T(b - Ax^k))$	32

כל שלושת המודלים יהיו untied per layer: לכל שכבה יהיה scalar משלה. זה מדגים deep unfolding אמיתי עם פרמטרים שונים לפי איטרציה.

שיטות אימון שנשווה

שיטה	רעיון	משמעות
L1	Final loss only	מאמנים לפי הפלט הסופי בלבד
L2	Intermediate supervision	מוסיפים loss גם לשכבות ביניים
L3	Greedy / sequential training	מאמנים שכבה-שכבה ומקפאים שכבות קודמות

לפני שימוש ב-L3 צריך לתקן את train_sequential, כי כרגע הוא לא מקפא מחדש שכבות קודמות בכל שלב.

9. מה חשוב לומר בהקלטה

- הבעיה היא sparse recovery: שחזור אות דליל ממדידות לינאריות מעטות.
- ISTA/FISTA הם baselines קלאסיים ללא למידה.

3. LISTA הוא ISTA unfolded: כל איטרציה הופכת לשכבה, והמשקלים נלמדים.
4. ALISTA/HyperLISTA מראים את הרעיון של MBDL: לשמור הרבה מבנה מודלי וללמוד מעט פרמטרים.
5. NMSE dB הוא מדד איכות השחזור המרכזי; ערך שלילי יותר הוא טוב יותר.
6. זמן ריצה ומספר פרמטרים הם מדדים שונים: מעט פרמטרים לא מבטיחים זמן ריצה קצר.
7. ההמשך שלנו הוא לא רק להריץ עוד דאטה, אלא לחקור אילו רכיבים מתוך ISTA כדאי ללמוד ואיך כדאי לאמן unfolded optimizer.

10. סטטוס עדכונים טכניים

- תיקנו את אתחול LISTA כך שיתאים ל-ISTA.
- ניקינו את Notebook 02 והשארנו אותו כניסוי final-loss נקי.
- התחלנו לעבוד עם workflow מסודר של branch לכל שינוי, בדיקה, merge, commit ל-main.
- הכלל להמשך: לפני push ל-main מריצים לפחות compile/test רלוונטי, ואם מדובר ב-notebook אז בודקים שהתוצאות הגיוניות.

Part B - English Summary

1. Current Project Snapshot

The project studies Deep Unfolding for sparse recovery. The starting point is ISTA for the LASSO / sparse recovery problem. The goal is to convert iterations of a classical optimizer into trainable architectures while preserving the model-based structure of the problem.

So far, we built a clean synthetic sparse-recovery experimental setup, aligned LISTA initialization with ISTA, cleaned Notebook 02 into a clean main comparison, and merged these changes into main. The next stage is to study the ISTA unfolding design space: different training methods and new ISTA-derived models that learn only selected components of the ISTA update.

2. Base Problem

We use the linear measurement model:

$$\mathbf{b} = \mathbf{A} \mathbf{x} + \boldsymbol{\varepsilon}$$

where $\mathbf{x}$ is sparse, $\mathbf{A}$ is the sensing matrix, and $\boldsymbol{\varepsilon}$ is measurement noise. In the noiseless case, $\boldsymbol{\varepsilon} = \mathbf{0}$. The goal is to reconstruct $\mathbf{x}$ from $\mathbf{b}$.

The classical LASSO formulation is:

$$\min_{\mathbf{x}} 0.5 \|\mathbf{b} - \mathbf{A} \mathbf{x}\|^2 + \lambda \|\mathbf{x}\|_1$$

ISTA solves this objective by repeating a gradient step followed by soft-thresholding:

$$\mathbf{x}^{k+1} = \mathbf{S}_{\{\lambda/L\}}(\mathbf{x}^k + (1/L) \mathbf{A}^T(\mathbf{b} - \mathbf{A} \mathbf{x}^k))$$

3. Connection to the Course

The course frames model-based methods and deep learning as two ends of a spectrum. In our project:

- **ISTA/FISTA** are classical model-based baselines with no training.
- **LISTA** unfolds ISTA into a layered trainable architecture with learned matrices and thresholds.
- **ALISTA** preserves more model-based structure by using an analytically computed correction matrix and learning only a few scalars.
- **HyperLISTA** is an even more compact adaptive version controlled by very few parameters.

The project therefore asks a core MBDL question: how much model structure should be preserved, and how much trainable flexibility should be introduced?

4. What We Have Done So Far

Notebook 01 - Classical Sparse Recovery Baselines

Notebook 01 creates a clean classical baseline. We run ISTA and FISTA on synthetic sparse recovery and perform an initial λ sweep to choose a reasonable baseline setting.

This notebook defines the reference point: what can be achieved without learning.

Notebook 02 - Main Synthetic Deep-Unrolling Comparison

Notebook 02 is the first main MBDL experiment. It compares ISTA/FISTA against LISTA, ALISTA, and HyperLISTA on the same synthetic sparse-recovery task.

We cleaned it so that:

- All relevant models use the same `LAM = 0.1`.
- LISTA is initialized from the ISTA parameterization.
- The main LISTA training uses final loss only: `intermediate_weight = 0.0`.
- Notebook-specific outputs and checkpoints are organized under the notebook-specific results folder.

LISTA Initialization Fix

We updated LISTA so that its initial threshold matches ISTA:

$$\theta_0 = \lambda / L$$

Since the code represents the threshold as `theta = softplus(log_theta)`, we set:

$$\log_theta = \text{softplus}^{-1}(\lambda/L)$$

Therefore the effective initial threshold is exactly `theta =  $\lambda/L$` , matching ISTA.

5. Notebook 02 Results

Model	NMSE dB	Runtime	Learned Parameters	Interpretation
ISTA	-5.306	8.132 ms	0	Classical baseline
FISTA	-10.959	7.895 ms	0	Accelerated classical baseline
LISTA	-20.866	8.576 ms	6,000,016	Highly flexible learned optimizer
ALISTA	-29.763	18.633 ms	32	Strong model structure with few learned scalars
HyperLISTA	-61.386	47.034 ms	3	Extremely compact and strong on the synthetic task

Main Message from the Table

Moving from classical ISTA/FISTA to unfolded trainable models significantly improves reconstruction quality. ALISTA and HyperLISTA show that large gains do not necessarily require millions of parameters; a strong model-based inductive bias can make very compact models effective.

Why is LISTA weak in intermediate layers but better at the end?

In Notebook 02, LISTA was trained using final loss only:

$$\text{Loss} = ||x - \hat{x}^{\{16\}}||^2$$

There is no explicit penalty on intermediate outputs. Therefore, intermediate layers are not required to be good reconstructions; they may learn internal representations that mainly help the final layer. ALISTA and HyperLISTA are more constrained and algorithmic, so their layers behave more like refinement steps.

Why are ALISTA and HyperLISTA slower despite having fewer parameters?

Parameter count is not the same as runtime. LISTA mostly performs dense matrix multiplications, which are highly optimized. ALISTA and HyperLISTA include residual updates, specialized thresholding, support selection, and adaptive computations. Thus, they are parameter-efficient but not necessarily runtime-minimal.

6. Main Metrics

Metric	Meaning	Why it matters
NMSE	Normalized reconstruction error: $\ x - \hat{x}\ ^2 / \ x\ ^2$	Main reconstruction-quality metric
NMSE dB	$10 \log_{10}(\text{NMSE})$	More negative is better
NMSE per layer	NMSE after each layer / iteration	Shows convergence behavior and internal dynamics
Runtime	Inference time	Measures practical computational cost
Number of parameters	Number of trainable parameters	Measures the balance between model-based structure and learned flexibility

7. Instructor Feedback and Its Implications

We reported that the project investigates Deep Unrolling for sparse recovery and later image recovery. The instructor approved using MNIST/FashionMNIST and noted that DCT or wavelets are unnecessary because these datasets are already sparse enough in the pixel domain. He also recommended comparing the different training methods discussed in class for unfolded optimizers.

Therefore the next key project directions are:

1. Compare training methods for unfolded optimizers.
2. Move to FashionMNIST/MNIST in the pixel domain with a clean train/validation/test split.

8. Next Planned Stage

Notebook 03 - ISTA Unfolding Design Space

We plan to turn the current training-method notebook into the third main experiment, likely named:

03_ista_unfolding_design_space.ipynb

Its purpose will be to show that we are not only running existing models; we are analyzing which components of ISTA should be learned.

Three New ISTA-Derived Models

Model	Learned Components	Conceptual Update	Parameters for K=16
ThresholdISTA	Per-layer threshold only	$\hat{x}^{k+1} = S_{\{\theta_k\}}(\hat{x}^k + (1/L)A^T(b - A\hat{x}^k))$	16
StepISTA	Per-layer step size only	$\hat{x}^{k+1} = S_{\{\lambda\gamma_k\}}(\hat{x}^k + \gamma_k A^T(b - A\hat{x}^k))$	16
StepThresholdISTA	Step size + threshold	$\hat{x}^{k+1} = S_{\{\theta_k\}}(\hat{x}^k + \gamma_k A^T(b - A\hat{x}^k))$	32

All three models will be untied per layer: each layer has its own scalar parameter. This demonstrates genuine deep unfolding with iteration-dependent learned parameters.

Training Methods to Compare

Method	Idea	Meaning
L1	Final loss only	Train using only the final output
L2	Intermediate supervision	Add loss terms on intermediate layers
L3	Greedy / sequential training	Train layer by layer while freezing previous layers

Before using L3, we need to fix `train_sequential`, because it currently does not properly refreeze previous layers at each stage.

9. What to Say in the Recording

1. The task is sparse recovery: reconstructing a sparse signal from compressed linear measurements.
2. ISTA and FISTA are classical non-learned baselines.
3. LISTA is an unfolded version of ISTA: each iteration becomes a layer, and the update matrices become trainable.
4. ALISTA and HyperLISTA demonstrate the MBDL idea: preserve strong model structure and learn only a small number of parameters.
5. NMSE dB is the main reconstruction-quality metric; more negative values are better.
6. Runtime and parameter count measure different things: fewer parameters do not always imply faster inference.
7. The next stage is to study not only more data, but the design space of ISTA unfolding: which components to learn and how to train the unfolded optimizer.

10. Technical Status

- LISTA initialization was aligned with ISTA.
- Notebook 02 was cleaned into a final-loss-only main comparison.
- We adopted a structured Git workflow: one branch per methodological change, test before commit, then merge into main.
- Going forward, before pushing to main, we will run at least a compile/test relevant to the change. For notebook changes, we will also verify that the resulting metrics are sensible.